Blockchain Fundamentals

Dr. Ahlad Kumar

NFSU Gandhinagar

October 4, 2025

Dr. Ahlad Kumar

Advantages of Blockchain over Conventional Distributed Databases

Blockchain is also a type of distributed database, but it brings unique properties that conventional distributed databases usually don't have.

Dr. Ahlad Kumar

Decentralization

- **Blockchain:** No single authority controls the database. Every node has a copy of the ledger and participates in consensus.
- **Conventional DB:** Controlled by a central administrator (e.g., a company's database cluster).
- **Advantage:** Eliminates single point of failure, reduces reliance on trust in one party.

Dr. Ahlad Kumar

Immutability (Tamper Resistance)

- **Blockchain:** Once data is added and confirmed, it cannot be altered without consensus and huge computational effort.
- **Conventional DB:** Administrators can modify or delete records easily.
- **Advantage:** Ensures data integrity, useful for audits, digital forensics, and trustless systems.

Dr. Ahlad Kumar

Consensus Mechanism

- **Blockchain:** Uses PoW, PoS, or BFT to agree on ledger state.
- **Conventional DB:** Uses master-slave replication, quorum, or Paxos/Raft, assuming trusted parties.
- **Advantage:** Blockchain can operate in **untrusted environments** with malicious actors.

Dr. Ahlad Kumar

Transparency & Auditability

- **Blockchain:** All transactions are verifiable publicly (or within consortium/private chain).
- **Conventional DB:** Requires logs and trust in administrators for audit.
- **Advantage:** Anyone can independently verify history $\rightarrow$ increases accountability.

Dr. Ahlad Kumar

Data Provenance

- **Blockchain:** Each record is linked cryptographically to the previous block.
- **Conventional DB:** Logs can be tampered with; no guaranteed traceability.
- **Advantage:** Strong chain-of-custody guarantees, useful in supply chain, healthcare, finance.

Dr. Ahlad Kumar

Fault Tolerance

- **Blockchain:** Survives even if some nodes are offline or malicious.
- **Conventional DB:** Designed for high availability but assumes trusted operators.
- **Advantage:** Handles Byzantine faults better (dishonest/malicious nodes).

Dr. Ahlad Kumar

Trustless Environment

- **Blockchain:** Trust is enforced via cryptography and consensus.
- **Conventional DB:** Requires trust in database admins/organization.
- **Advantage:** Enables peer-to-peer systems (e.g., Bitcoin) without central authority.

Dr. Ahlad Kumar

Summary

- **Conventional DB:** Best for speed, efficiency, and trusted environments.
- **Blockchain:** Best for tamper-proofing, decentralization, and operation among untrusted parties.

Blockchain is not always faster, but it ensures trust where trust is missing.

Dr. Ahlad Kumar

Blockchain vs Conventional Distributed Database

Blockchain Advantages

- **Decentralization:** No single control, avoids single point of failure
- **Immutability:** Records cannot be tampered once confirmed
- **Consensus:** Works even in untrusted environments (PoW, PoS, BFT)
- **Transparency:** Publicly verifiable transactions
- **Data Provenance:** Cryptographic linking ensures traceability
- **Fault Tolerance:** Resilient to malicious or offline nodes
- **Trustless:** Eliminates need to trust administrators

Conventional Distributed DB

- Controlled by central authority or admins
- Records can be modified or deleted
- Relies on trusted replication/quorum mechanisms
- Limited transparency, requires audit logs
- No inherent cryptographic data lineage
- Assumes nodes are honest/trusted
- Requires trust in database operator

Dr. Ahlad Kumar

Blockchain is preferred when tamper-resistance, decentralization, and trustless operation are required.

Life of a Blockchain Application

When we say "life of a blockchain application", we usually mean the lifecycle of how a blockchain-based system (say Bitcoin, Ethereum, or a supply-chain DApp) comes into existence, runs, and evolves.

Dr. Ahlad Kumar

What is a Blockchain Application?

- A blockchain application (or DApp – Decentralized Application) is software that uses blockchain as its backend database instead of (or along with) traditional databases.
- **Examples:**
  - Bitcoin → a payment application built on blockchain
  - Supply chain tracking → blockchain records every step of product movement
  - NFTs → blockchain proves ownership of digital assets
- Unlike normal apps that store data in MySQL/Oracle, blockchain apps store data on a distributed ledger where records are immutable and consensus-driven.

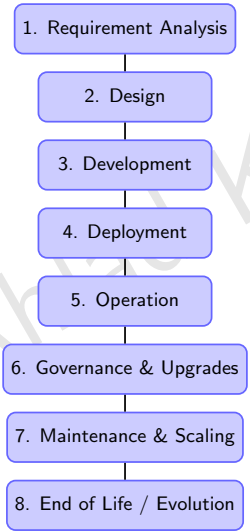
Dr. Ahad Kumar

Lifecycle (Life of a Blockchain Application)

The lifecycle of a blockchain application follows several stages, from idea to operation and evolution.

Dr. Ahlad Kumar

Lifecycle of a Blockchain Application



Dr. Ashok Kumar

1. Requirement Analysis & Use-Case Selection

- Decide: Why do we need blockchain?
- If decentralization, transparency, or tamper-proof records are required $\rightarrow$ blockchain makes sense.
- Example: Voting system, land records, healthcare data sharing.

Dr. Ahlad Kumar

2. Design

- Choose the type of blockchain:
  - Public (Bitcoin, Ethereum) $\rightarrow$ open to all
  - Private/Consortium $\rightarrow$ controlled by organizations
- Define consensus mechanism (PoW, PoS, PBFT, etc.)
- Decide smart contracts or transaction logic

Dr. Ahad Kumar

3. Development

- Build smart contracts (if applicable, e.g., in Solidity for Ethereum).
- Create frontend and backend logic.
- Integrate wallets, cryptography, and APIs.

Dr. Ahlad Kumar

4. Deployment

- Nodes are set up across the network.
- Genesis block is created (the “birth” of the blockchain).
- Application starts recording transactions onto the blockchain.

Dr. Ahlad Kumar

5. Operation (Running Phase)

- Users interact with the app (send coins, record supply chain data, cast votes).
- Transactions are broadcast to the network.
- Miners/validators verify, order, and add transactions to blocks.
- Ledger is updated across all nodes.

Dr. Ahlad Kumar

6. Governance & Upgrades

- Bugs, performance issues, or new features may require changes.
- Hard fork / soft fork $\rightarrow$ blockchain rules are updated.
- Example: Ethereum hard-fork after DAO hack.

Dr. Ahad Kumar

7. Maintenance & Scaling

- Handle growing user base and transaction volume.
- Optimize block size, consensus, or layer-2 solutions (like Lightning Network for Bitcoin).

Dr. Ahlad Kumar

8. End of Life (or Evolution)

- Some blockchains/applications may fail (low adoption, high cost).
- Others evolve into new versions via forks.
- Example: Bitcoin Cash forked from Bitcoin; Ethereum 2.0 upgraded to Proof of Stake.

Dr. Ahlad Kumar

Summary

- A blockchain application is software that runs on top of a blockchain ledger.
- Its life goes from **idea → design → development → deployment → operation → governance → scaling → end/evolution**.
- Unlike conventional apps, its core strength lies in decentralization, immutability, and trustless operation.

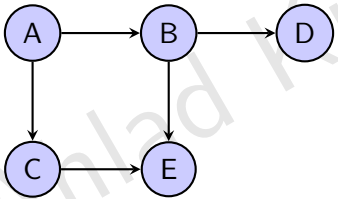
Dr. Ahlad Kumar

What is a DAG?

- **Directed** – edges have a specific direction (arrows).
- **Acyclic** – contains no cycles (no way back to the same node).
- **Graph** – collection of nodes (vertices) and directed edges.
- Flows only forward, never loops back.

Dr. Ahlad Kumar

DAG Example



Dr. Anlad Kumar

Properties of DAG

- No cycles – cannot return to the same node following edge directions.
- Supports **Topological Ordering** of nodes.
- Represents **Partial Ordering** (dependencies).
- May have multiple **roots** (start nodes) and **leaves** (end nodes).
- Transitive property: If $A \rightarrow B$ and $B \rightarrow C$, then A precedes C.
- Used for workflows, scheduling, Git history, and blockchain (IOTA, Hedera).

Dr. Ahlad Kumar

DAG as an Alternative to Blockchain

- Traditional blockchain: a **linear chain** of blocks.
- DAG-based systems: a **graph structure** of transactions.
- Each transaction references (approves) previous ones.
- Removes the need for fixed block intervals.

Dr. Ahlad Kumar

Properties and Benefits of DAG in Blockchain

- **Parallelism & Scalability** – multiple transactions can be confirmed simultaneously.
- **No Mining / Low Fees** – users validate transactions by approving others.
- **Security through Confirmations** – more approvals = stronger validity.
- **Efficiency** – lightweight and suitable for IoT and micro-payments.

Dr. Ahlad Kumar

Blockchain vs DAG

Feature	Blockchain	DAG
Structure	Linear chain of blocks	Graph of transactions
Consensus	Mining (PoW/PoS)	Transactions approve past ones
Speed	Slower (block time)	Faster, parallel confirmations
Fees	Usually required	Very low or zero
Scalability	Limited	Highly scalable

Dr. Ahlad Kumar

Real-World DAG Blockchain Examples

- **IOTA (Tangle)** – IoT payments, fee-less micro-transactions.
- **Nano** – block-lattice DAG, instant payments.
- **Hedera Hashgraph** – gossip protocol on DAG for consensus.

Dr. Ahlad Kumar

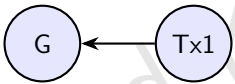
DAG growth — step-by-step



Step 1: Genesis exists (G). New transactions will reference earlier ones.

Dr. Ahlad Kumar

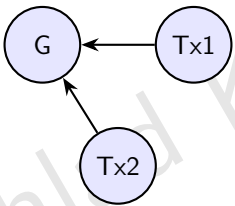
DAG growth — step-by-step



Step 2: Tx1 issued and references G. Tx1 is a tip (unconfirmed).

Dr. Ahlad Kumar

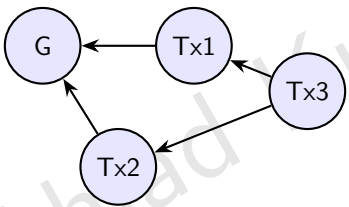
DAG growth — step-by-step



Step 3: Tx2 issued, also references G. Tips now: Tx1 and Tx2.

Dr. Ahlad Kumar

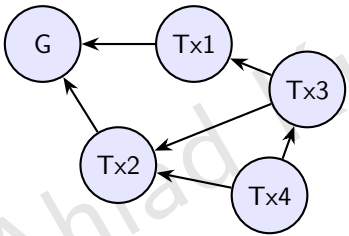
DAG growth — step-by-step



Step 4: Tx3 approves Tx1 & Tx2 directly — both gain confirmations.

Dr. Ahad Kumar

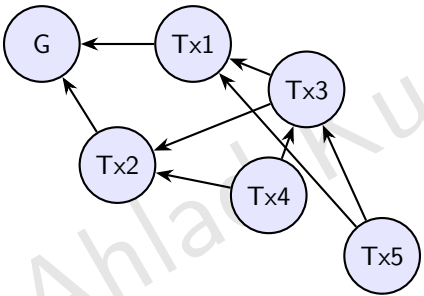
DAG growth — step-by-step



Step 5: Tx4 approves Tx2 & Tx3 — confirmations spread further (indirect confirmations).

Dr. Ahlad Kumar

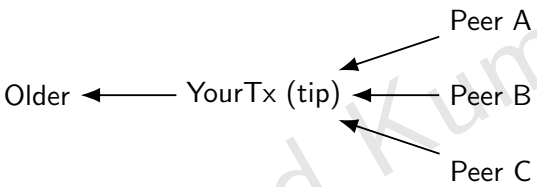
DAG growth — step-by-step



Step 6: Tx5 approves Tx3 & Tx1. Already-approved nodes get extra direct confirmations; cumulative weight increases, pushing toward finality.

Dr. Ahlad Kumar

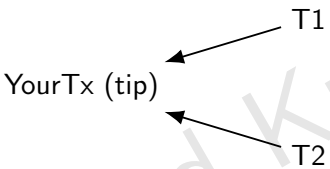
Broadcast



Dr. Ahlad Kumar

- When you send a transaction it is broadcast (gossiped) to peers and becomes a new **tip** (an unapproved leaf).
- Peers can see it immediately, but visibility alone is **not** strong proof of irreversibility.
- Broadcast = "seen", not final.

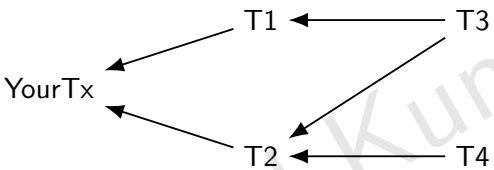
Direct confirmation = someone references you



- If the next transaction(s) explicitly reference/approve your tx, this is a **direct confirmation**.
- Direct confirmations increase confidence, but one or two direct approvals usually do not guarantee finality.

Dr. Ahlad Kumar

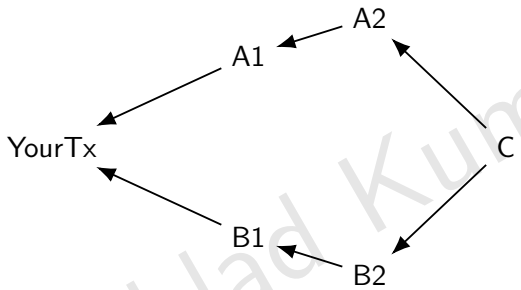
Indirect confirmation = approvals-of-approvals



- Later transactions can approve transactions (like T1/T2) that already approve your tx.
- These are **indirect confirmations** (approvals-of-approvals).
- Example: Tx5 approves Tx3, and Tx3 already approved your tx → your confirmation weight increases.
- This builds a confirmation chain behind your tx.

Dr. Ahlad Kumar

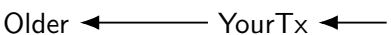
Cumulative weight → confidence



Dr. Ahlad Kumar

- Protocols measure **cumulative weight** or a **confirmation confidence** based on how many honest transactions (directly or indirectly) reference your tx.
- As cumulative weight grows, nodes' confidence that the tx is permanent increases (e.g., "99.9% chance not reversed").
- Higher network activity → faster growth of confidence.

Finality threshold depends on the protocol



- A transaction is considered **confirmed / final** once its confirmation confidence crosses a protocol-defined threshold.
- Different approaches:
  - **Probabilistic (IOTA/Tangle)** — final when cumulative weight makes reversal extremely unlikely.
  - **Representative voting (Nano)** — representatives vote; when they commit, tx is final.
- In every case: the deeper (more approvals) your tx is buried, the harder it is to reverse → effectively final.

Dr. Ahlad Kumar

Summary (short)

- **Broadcast**: tx becomes a tip and is seen — not final.
- **Direct confirmation**: next txs reference your tx $\rightarrow$ initial confirmations.
- **Indirect confirmation**: approvals-of-approvals build cumulative weight.
- **Confidence**: cumulative weight $\rightarrow$ confirmation probability.
- **Finality**: protocol-dependent threshold; deeper burial stronger finality.

Dr. Ahlad Kumar

Introduction to Soft Fork

- A **Soft Fork** is a change in blockchain rules that is **backward compatible**.
- Old nodes (not upgraded) can still accept new blocks as valid.
- New nodes (upgraded) both *accept* and *validate* the stricter rules.
- Only one chain continues — no permanent split (unlike a hard fork).
- Example: **Bitcoin SegWit (2017)**.

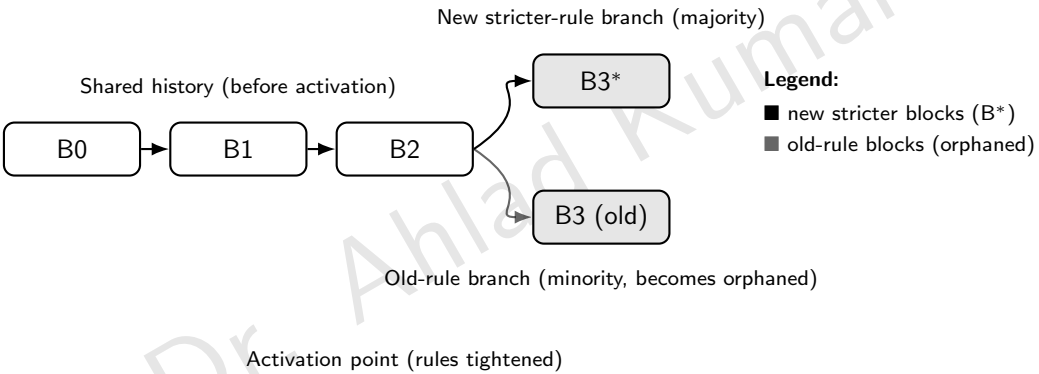
Dr. Ahlad Kumar

Soft Fork: Example

- Imagine a library where books could be borrowed for up to **30 days**.
- New stricter rule: books can now be borrowed only for **15 days**.
- Old members (who don't know about the new rule) still see a 15-day borrow as valid — since it is within the old 30-day limit.
- New members (aware of the new rule) check strictly: no one can borrow beyond 15 days.
- Result: everyone still uses the same library system, no permanent split.

Dr. Ahlad Kumar

Soft Fork

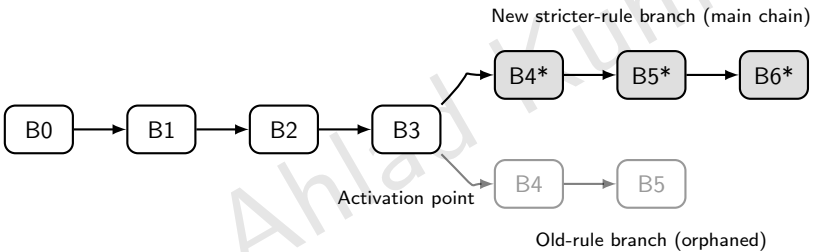


Dr. Ahlad Kumar

- **Shared history:** Blocks B0 $\rightarrow$ B2 are common to everyone before the upgrade (no disagreement).
- **Activation point:** A rule is tightened (soft fork). After this, miners can produce blocks under the new stricter rule.
- **Temporary branch:** Both a *new stricter* branch and a *old-rule* branch can appear briefly if not all miners upgrade.
- **Majority wins:** If most miners enforce the new rule, their branch grows longer and becomes the *main chain*. The other branch's blocks are orphaned (cut off).
- **Old nodes vs new nodes:**
  - **Old nodes:** *Accept only* — they do not enforce the new checks but still treat new blocks as valid (backward compatibility).
  - **New nodes:** *Accept + Validate* — they enforce the additional stricter rules.
- **Result:** No permanent split (unlike a hard fork). Network converges to one chain under the stricter rules — provided majority support.
- **Practical note:** Example — Bitcoin's SegWit (2017) was deployed as a soft fork: old nodes accepted SegWit blocks while upgraded nodes validated the new witness rules.

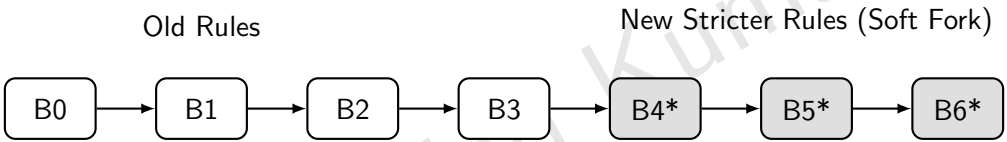
Dr. Ahlad Kumar

Soft Fork — Temporary Branching



Dr. Ahlad Kumar

Soft Fork in Blockchain



Old nodes still accept new blocks

(backward compatible)

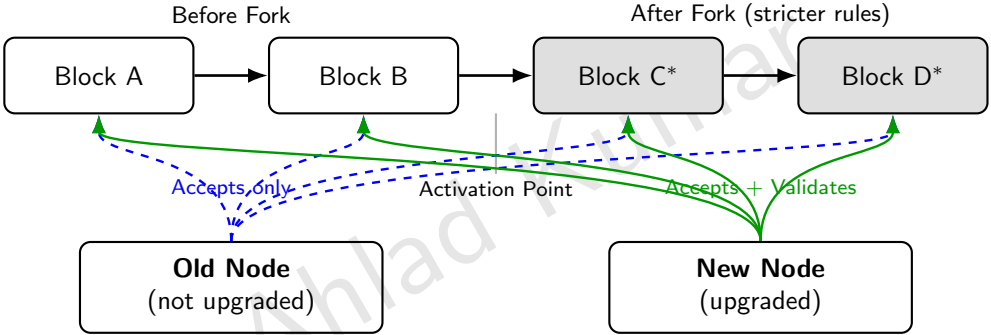
Dr. Ahlad Kumar

Soft Fork — Temporary Branching and Convergence

- Activation tightens rules $\rightarrow$ two branches can appear briefly.
- If the majority of miners enforce the new rule, their branch becomes longer and the other branch is orphaned (cut off).
- Result: a single continuing chain under the stricter rules — no permanent split.

Dr. Ahlad Kumar

Soft Fork — Old vs New Node Validation



Old nodes keep the chain running but do not enforce the new rule.
New nodes enforce the stricter rule in addition to accepting the block.

- New node: accepts + enforces new rules
- Old node: accepts but does not enforce new rules

Dr. Ahlad Kumar

Soft Fork : What the Diagram Shows

- **Timeline of blocks:**
  - Block A and B created before fork (old rules).
  - Block C* and D* created after fork (new stricter rules).
- **Activation point:** the moment when stricter rules come into effect.
- **Old Node (not upgraded):**
  - Still *accepts* all blocks, including new ones.
  - Does not enforce the new stricter rules.
  - Role: keeps the chain running with backward compatibility.
- **New Node (upgraded):**
  - *Accepts* + *validates* blocks.
  - Enforces stricter rules for blocks after the fork.
  - Ensures security and correctness under the upgrade.
- **Key takeaway:**
  - Both old and new nodes stay on the same chain.
  - Soft fork is backward compatible — no permanent split.

Dr Ahlad Kumar

Bitcoin Soft Fork — SegWit (2017)

- **Segregated Witness (SegWit)** introduced in 2017 via BIP 141.
- Major goals:
  - Fix **transaction malleability** (allowed Lightning Network to be built).
  - Increase effective **block capacity**.
- **How it worked:**
  - Signature data ("witness") was moved outside the base block.
  - Max **block weight** = 4 MB.
  - In practice, effective block size increased from $\sim$1 MB $\rightarrow$ up to $\sim$4 MB.
- **Backward compatible:** old nodes saw SegWit transactions as valid (treated witness data as "anyone-can-spend").
- Considered the most important Bitcoin **soft fork**.

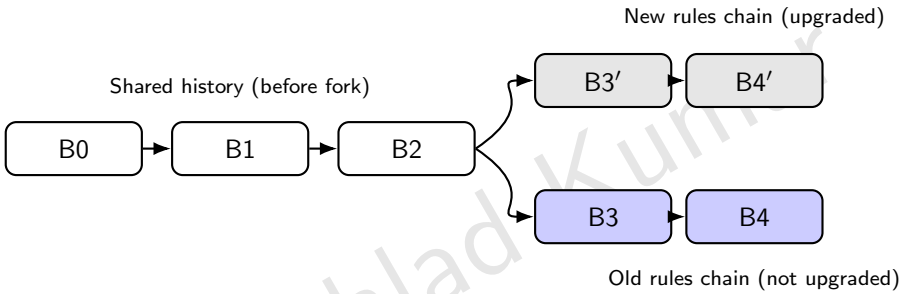
Dr. Ahlad Kumar

Ethereum Soft Fork — EIP-150 (Tangerine Whistle, 2016)

- **EIP-150** was part of the Tangerine Whistle upgrade (Oct 2016).
- Motivation: prevent **DoS attacks** caused by underpriced opcodes.
- **What changed:**
  - Increased gas costs for certain opcodes:
    - ⋆ e.g., `CALL`, `EXTCODESIZE`, `BALANCE`.
  - Reduced potential for attackers to spam the network cheaply.
- **Backward compatible:**
  - Transactions still valid under old interpretation.
  - Just cost more gas to execute after the fork.
- Improved overall **network security and performance**.

Dr. Ahlad Kumar

Hard Fork



Dr. Ahlad Kumar

Old chain (B, old rules)
New chain (B′, new rules)

- **Shared history:** Blocks B0 $\rightarrow$ B2 are common to everyone before the fork.
- **Activation point:** Rules are changed in a way that is **not backward compatible**.
- **Permanent split:** Two chains continue after the fork:
  - New rules chain (upgraded nodes follow this).
  - Old rules chain (non-upgraded nodes keep following this).
- **No convergence:** Unlike a soft fork, both chains remain valid $\rightarrow$ two separate blockchains.
- **Old vs new nodes:**
  - Old nodes **reject** blocks from the new chain (they don't recognize the new rules).
  - New nodes **reject** blocks from the old chain.
- **Result:** Permanent network split into two blockchains.
- **Examples:**
  - Bitcoin vs Bitcoin Cash (2017).
  - Ethereum vs Ethereum Classic (2016).

Dr. Ahlad Kumar

Hard Fork — Fork Interface and Records

- **Fork Interface (B2 → B3' and B3):**
  - Up to Block B2: all nodes share the same history.
  - At activation (after B2): a new rule is introduced.
  - Upgraded nodes build **B3'** under the new rules.
  - Non-upgraded nodes build **B3** under the old rules.
  - Result: chain splits into two valid histories from the same parent (B2).

- **How Records Are Maintained:**
  - **Chain A (new rules):** B3', B4', ... Only upgraded nodes recognize these blocks. Old nodes reject them.
  - **Chain B (old rules):** B3, B4, ... Only old nodes recognize these blocks. Upgraded nodes reject them.
  - Both chains maintain valid records, but for different participants.
  - Ledgers drift apart permanently → no single shared history.

Dr. Ahlad Kumar

Real Hard Forks — Bitcoin and Ethereum

- **Bitcoin Hard Fork (2017) — Bitcoin Cash**
  - Occurred at Block 478,558 (Aug 2017).
  - Bitcoin (BTC): kept SegWit, effective block size $\sim$1 MB (block weight).
  - Bitcoin Cash (BCH): removed SegWit, increased block size to 8 MB.
  - Result: Two separate currencies, BTC and BCH.

- **Ethereum Hard Fork (2016) — DAO Hack**
  - Occurred at Block 1,920,000 (July 2016).
  - Ethereum (ETH): reverted DAO hack, funds returned to investors.
  - Ethereum Classic (ETC): kept original history without rollback.
  - Result: Two separate chains, ETH and ETC, coexisting permanently.

Key Point: Hard forks create **permanent splits**, where both chains continue independently with different rule sets.

Dr. Akhilesh Kumar

Public vs Private Blockchain

Feature	Public Blockchain	Private Blockchain
Accessibility	Open to anyone; permissionless	Restricted; only authorized participants (permissioned)
Examples	Bitcoin, Ethereum	Hyperledger Fabric, R3 Corda
Control	Fully decentralized, no single authority	Controlled by an organization or consortium
Consensus Mechanism	PoW, PoS, other open protocols	PBFT, RAFT, or other lightweight protocols
Transaction Speed	Slower (many nodes, complex consensus)	Faster (fewer nodes, efficient consensus)
Transparency	Full transparency; everyone can view	Limited to authorized participants
Security	Very secure (large distributed network)	Depends on trust in central authority
Immutability	High; nearly impossible to alter records	Moderate; records can be altered if allowed
Cost	High (mining, energy intensive)	Lower (efficient consensus, no mining)
Use Cases	Cryptocurrencies, DeFi, dApps	Enterprise, supply chain, banking, healthcare

Dr. Ahlad Kumar

Distributed Consensus in Blockchain

Definition:
Distributed consensus ensures that all nodes in a blockchain network agree on the same valid transaction history without relying on a central authority.

Why it is Important:

- Ensures all nodes maintain the same version of the ledger
- Prevents double spending and fraudulent transactions
- Achieves trust among untrusted participants
- Decides which block is added next to the blockchain

Property	**Explanation**
Agreement	Honest nodes agree on the same block-/ledger state
Validity	Only valid transactions are accepted
Fault Tolerance	Works even if some nodes fail or act maliciously
Finality	Once confirmed, blocks are irreversible

Dr. Ahlad Kumar

Chain Policy in Blockchain

Definition:
A *chain policy* is the set of rules and governance mechanisms that define how a blockchain operates, including validation, membership, transactions, and upgrades.

Key Components:

- **Consensus Rules:** How new blocks are validated (PoW, PoS, PBFT, RAFT).
- **Membership Rules:** Who can join the network (public vs private blockchains).
- **Transaction Policies:** Allowed transaction types, block size, gas fees.
- **Governance Policies:** How protocol upgrades, forks, and disputes are resolved.
- **Security & Privacy Policies:** Data visibility, encryption, identity management.

Examples:

- Bitcoin: Block size limit, block interval (10 min), difficulty adjustment.
- Hyperledger Fabric: Endorsement policies, access control for private networks.

Dr. Ahlad Kumar

Anonymity in Blockchain

Definition:
Anonymity in blockchain means users can transact without revealing their real-world identity. Instead, they are represented by cryptographic addresses (public keys).

How It Works:

- Transactions are linked to wallet addresses, not names.
- Example: Instead of *Alice* $\rightarrow$ *Bob*, blockchain records `1FfmbHfn...` $\rightarrow$ `3J98t1Wp...`.

Advantages:

- Protects user privacy
- Prevents censorship and surveillance
- Encourages participation in open/public blockchains

Limitations:

- Not fully anonymous — most are pseudonymous
- Transaction patterns can still reveal identities
- Potential misuse (illegal trade, money laundering)

Dr. Ahlad Kumar

Mining Reward in Blockchain

Definition:
Mining reward = Block Subsidy (newly minted coins) + Transaction Fees.

Comparison of Bitcoin Mining Rewards

Year	Block Subsidy	Total Reward (incl. fees)
2009 (Genesis)	50 BTC per block	$\approx$ 50 BTC (very few tx fees)
2025 (Today)	3.125 BTC per block	$\approx$ 3.14 BTC (avg. 0.0138 BTC fees)

Key Insights:

- Rewards halve every $\sim$4 years (210,000 blocks).
- Started at 50 BTC $\rightarrow$ now 3.125 BTC (over 93% reduction).
- Transaction fees are a small portion today ($\sim$0.4% of block reward).
- By $\sim$2140, no subsidy will remain — miners will rely only on fees.

Dr Ahlad Kumar

Mining Mechanism in Blockchain

Definition:
Mining is the process of validating transactions, creating blocks, and securing the blockchain by solving cryptographic puzzles.

Steps in Mining:

1. **Transaction Broadcast:** Users send transactions, collected in the mempool.
2. **Block Formation:** Miner selects valid transactions and forms a candidate block.
3. **Proof of Work Puzzle:** Miner finds a nonce so the block hash meets the difficulty target.
4. **Consensus:** First valid block is broadcast; other nodes verify the solution.
5. **Block Addition:** Block is added to the blockchain.

Mining Reward:

- Block Subsidy (new coins) + Transaction Fees
- Bitcoin (2025): $\approx$ 3.14 BTC per block

Dr. Ahlad Kumar